CO1 – ASSIGNMENT 2

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

Student: Sudharsan R **Register No.:** 1912260284

Course: C Programming **Submission date:** 27-10-2026

TASK 1 – Organise the Program with Header Files and Conditional Compilation (3 Marks)

Program structure and header file

A well-organised C program separates declarations and shared constants into a header file, while the implementation and main() remain in .c files. Include guards prevent the same header from being processed more than once. Use **#include <header.h>** for system headers found through compiler include paths and **#include "header.h"** for a project header searched relative to the source file first.

File: billing.h

```
#ifndef BILLING_H
#define BILLING_H

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0LL
#define RATE2_PAISE 225LL
#define RATE3_PAISE 450LL
#define FIXED_CHARGE_PAISE 5000LL

long long compute_bill(int units);

#ifdef DEBUG
#include <stdio.h>
#define DEBUG_PRINT(...) \
    do { fprintf(stderr, "[DEBUG] "); \
         fprintf(stderr, __VA_ARGS__); \
         fputc('\n', stderr); } while (0)
#else
#define DEBUG_PRINT(...) ((void)0)
#endif

#endif /* BILLING_H */
```

File: billing.c

```
#include "billing.h"

long long compute_bill(int units)
{
    long long paise = FIXED_CHARGE_PAISE;

    if (units <= SLAB1_UNITS) {
        paise += (long long)units * RATE1_PAISE;
    } else if (units <= SLAB2_UNITS) {
        paise += (long long)(units - SLAB1_UNITS) * RATE2_PAISE;
    } else {
        paise += (long long)(SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE;
        paise += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
    }
    return paise;
}
```

File: main.c

```
#include <stdio.h>
#include "billing.h"

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int count = (int)(sizeof tests / sizeof tests[0]);

    for (int i = 0; i < count; ++i) {
        long long bill = compute_bill(tests[i]);
        DEBUG_PRINT("units=%d, bill=%lld paise", tests[i], bill);
        printf("%d units -> Rs %lld.%02lld\n",
               tests[i], bill / 100, bill % 100);
    }
    return 0;
}
```

Compile and run:

```
gcc -std=c11 -Wall -Wextra -pedantic main.c billing.c -o billing
./billing
gcc -std=c11 -Wall -Wextra -pedantic -DDEBUG main.c billing.c -o billing_debug
./billing_debug
```

Normal build prints bills only. The DEBUG build additionally prints intermediate units and bill values to standard error. Conditional compilation removes the debug calls from the normal build, so there is no run-time check for DEBUG in every call.

Alternative and complexity

If debugging used a run-time variable such as if (debug) printf(...), each reached debug point performs a condition check, even when debugging is off. The extra time is O(1) per debug point and total overhead is O(k) for k points; the executable retains the debug strings and code. With #ifdef DEBUG, those statements are absent from a non-debug executable, reducing code size and removing the run-time branch. Include guards also prevent duplicate declarations and definitions caused by repeated inclusion.

TASK 2 – Read and Print Data Correctly (4 Marks)

Correct format specifiers and input validation

Use the format specifier that matches the argument type: %d for int, %u for unsigned int, %ld for long, %lld for long long, %f for double in printf, %lf for double in scanf, %c for char, and %s for a character array. A mismatched format specifier causes undefined behaviour, which may print incorrect values or corrupt data. For scanf, check the return value: it must equal the number of conversions expected.

```
#include <stdio.h>

int main(void)
{
    int units;
    long long bill_paise;

    printf("Enter units consumed: ");
    if (scanf("%d", &units) != 1 || units < 0) {
        fprintf(stderr, "Invalid units. Enter a non-negative integer.\n");
        return 1;
    }

    printf("Enter bill in paise: ");
    if (scanf("%lld", &bill_paise) != 1 || bill_paise < 0) {
        fprintf(stderr, "Invalid bill amount.\n");
        return 1;
    }

    printf("Units: %d\n", units);
    printf("Bill: Rs %lld.%02lld\n",
           bill_paise / 100, bill_paise % 100);
    return 0;
}
```

Example input: 150 units and 16250 paise. Output: Units: 150; Bill: Rs 162.50. The validation detects letters or missing values because scanf returns fewer conversions than requested. In production, fgets() plus strtol() is more robust for rejecting trailing junk and detecting range errors.

Alternative: fgets() and strtol()

Reading a line with fgets() allows the program to inspect the entire input before conversion. For a line of length L, reading and validating it takes O(L) time and the buffer uses O(L) space. scanf() is concise and works for simple controlled input, but recovery from malformed input can be harder; fgets() with strtol() makes it easier to detect trailing characters, empty input and out-of-range values. A fixed buffer should be large enough and the program should detect whether the line was truncated.

TASK 3 – Control Type Conversion and Precision (4 Marks)

Where conversions occur

C promotes small integer types such as char and short to int in most arithmetic expressions. When signed and unsigned values are combined, the usual arithmetic conversions can convert a negative signed value to a large unsigned value. Integer division discards the fractional part. Cast before multiplication when a wide result is needed, and use integer paise for money so every charge is exact to one paise.

```
#include <stdio.h>

int main(void)
{
    int units = 150;
    int paise_per_unit = 225;
    int whole_rupees = 16250 / 100;
    long long exact_paise = (long long)units * paise_per_unit;
    double rupees = exact_paise / 100.0;
```

```
    printf("Surprise 1: 5 / 2 = %d (integer division)\n", 5 / 2);
    printf("Surprise 2: (double)(5 / 2) = %.1f\n",
           (double)(5 / 2));
    printf("Surprise 3: (double)5 / 2 = %.1f\n",
           (double)5 / 2);
    printf("150 * 225 paise = %lld paise = Rs %.2f\n",
           exact_paise, rupees);
    printf("16250 / 100 as integer = %d rupees\n", whole_rupees);
    return 0;
}
```

Expected output:

```
Surprise 1: 5 / 2 = 2 (integer division)
Surprise 2: (double)(5 / 2) = 2.0
Surprise 3: (double)5 / 2 = 2.5
150 * 225 paise = 33750 paise = Rs 337.50
16250 / 100 as integer = 162 rupees
```

The three expressions differ because casting after integer division cannot restore the discarded fraction, while casting an operand before division makes the operation floating-point. The final line intentionally shows that integer division loses the 50 paise; use bill_paise / 100 and bill_paise % 100 when formatting a bill.

Casting and rounding rule

For large totals, cast before multiplication: (long long)units * rate_paise. Casting after a potentially overflowing int multiplication is too late. If a tariff calculation genuinely creates a fractional paise, use a documented rule such as round-half-up at the final conversion, rather than rounding each intermediate step. With integer paise slab rates, no fractional-paise rounding is needed.

Alternative and complexity

Integer-paise arithmetic takes O(1) time and O(1) space per bill and has no binary floating-point rounding error, provided the integer type does not overflow. Double arithmetic also takes O(1) time and O(1) space per bill, but values such as 0.1 are not represented exactly in binary. If n bills are accumulated in floating point, error can accumulate with n; the exact bound depends on magnitudes and operation order. Integer paise is preferred for this billing problem.

TASK 4 – Trace the Compilation Process (4 Marks)

Compilation stages and commands

The C build process has four main stages: preprocessing, compilation to assembly, assembling to object code, and linking object files and libraries into an executable.

```
gcc -E main.c > main.i       # preprocessed C
gcc -S main.c -o main.s     # assembly text
gcc -c main.c -o main.o     # object file
gcc -c billing.c -o billing.o
gcc main.o billing.o -o billing  # link executable
./billing
```

Preprocessing expands macros, includes header contents, and handles conditional compilation such as #ifdef DEBUG. Compilation translates C into assembly and checks syntax and types. Assembling creates object code. Linking resolves references such as compute_bill() across main.o and billing.o and adds required libraries.

Classifying errors

Error type	Example	When it appears / solution
Compile-time	Missing semicolon or undeclared variable	Compiler reports syntax/name error; fix source and rebuild.
Link-time	Call compute_bill() but omit billing.o	Linker reports undefined reference; compile and link billing.c too.
Run-time	Invalid input, division by zero, or out-of-bounds access	Program runs but fails or behaves incorrectly; validate inputs and test.

Warnings and how to resolve them

Compile with gcc -std=c11 -Wall -Wextra -Wconversion -pedantic. Warnings can identify suspicious conversions, unused variables, or format mismatches. Read each warning, confirm whether it reveals a real bug, then correct the type, add a justified cast, validate the input, or remove unused code. Do not silence warnings without understanding them.

Alternative: separate source files

With one source file, changing any part usually requires recompiling the entire file. With several .c files and a shared header, only changed source files need recompilation, then the object files are linked again. For a small project, the time saved is modest; for large projects it can be substantial. Separate object files use additional disk space, but each object contains only compiled code and data for its source file. If a shared header changes, every source file depending on it should be recompiled; a build tool such as make tracks these dependencies.

Suggested README evidence

Include your name and register number, compiler version (gcc --version), the exact commands above, screenshots or copied output from each task, and a short note explaining what each result verifies. Create one folder per task and commit progress separately to the required GitHub repository. Add your actual GitHub repository URL to the report before submission; it cannot be generated without creating the repository in your account.

Conclusion

The program is easier to maintain when shared declarations are placed in guarded headers, debug logging is controlled at compile time, input is validated, money is stored in paise, conversions are explicit, and compilation stages are tested separately. These practices improve correctness, portability and debugging.